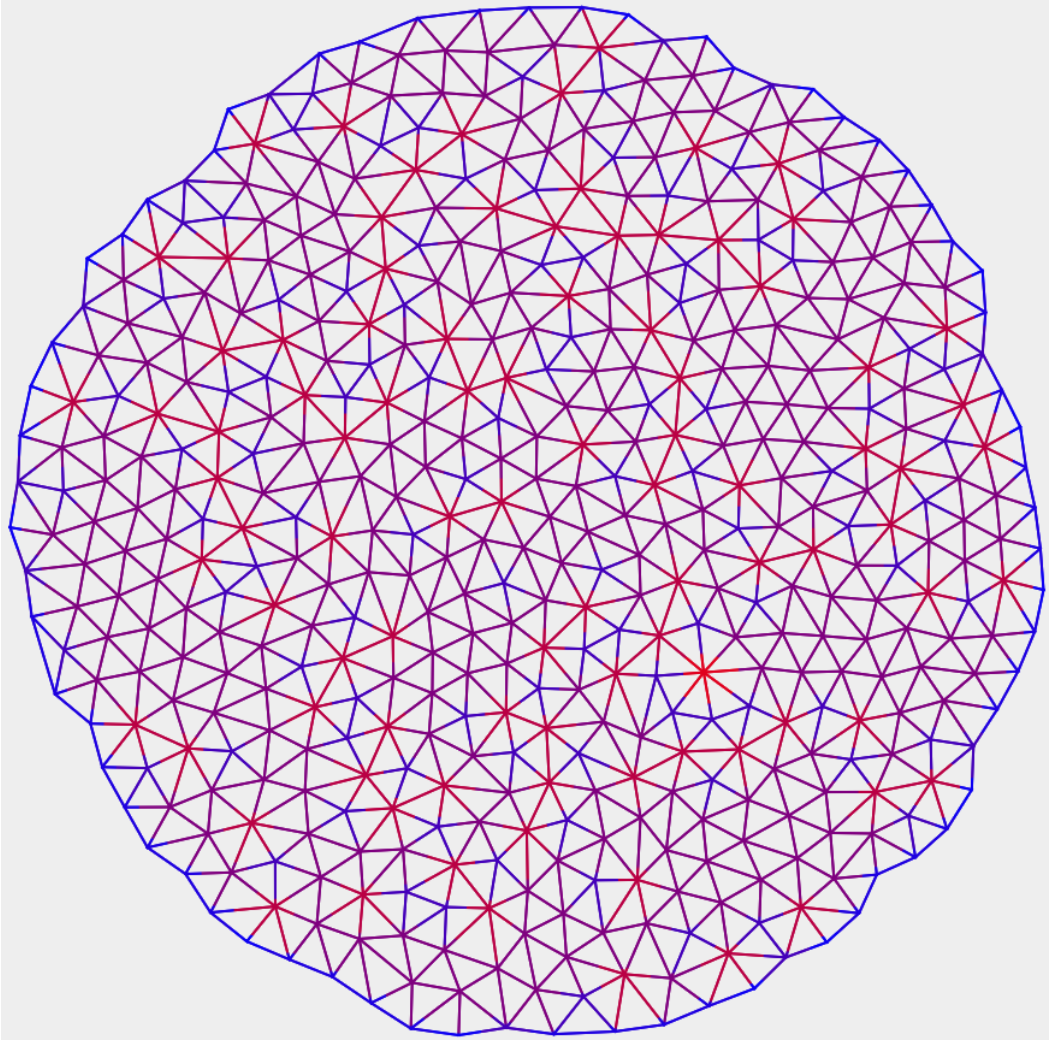
Canning for Amorphous Blob Computing

Internship Report

Author :
Lucas Labouret
M1 QDCS, Université Paris-Saclay
lucas.labouret@universite-paris-saclay.fr

Supervisor :
Frédéric Gruau
LISN
frederic.gruau@universite-paris-saclay.fr



Contents

Introduction

This report summarizes the work I performed at LISN under the supervision of Frédéric Gruau. It is the continuation of a research project I did under him last year my Licence degree's curriculum. It is part of a long term research project about the concept of "blob computing"[4].

My presence at LISN was spread over two periods : first, a TER from January to April 2025, then an internship from May to July 2025. During the TER, I performed research about the simulation of blob computing on classical computers, and wrote a GUI application that allows users to visualize and interact with my results. The internship then focused on improving and expanding on the results I obtained during the TER.

The program that I produced during this internship can be found **here**.

Note : Chapters 1 and 2 are identical the TER report I already submitted. I chose to include them anyway as they are necessary to understand the new chapter 3.

Chapter 1

Prerequisites

1.1 What is Blob Computing ?

To understand the exact subject of this TER, it is important to first introduce what blob computing means and how it works.

The ultimate goal of the project is to develop a new paradigm of computation that is inherently arbitrarily scalable through the use of *space* as a resource. This kind of computation is becoming increasingly useful, for example with the recent development of swarm robotic[7].

We aim to achieve this through the use of arbitrarily many processing elements (PEs) distributed through space and locally connected. These PEs, which can be severely lacking in power on their own, create a computing medium where virtual "blobs" can form and evolve. These blobs are the main primitive we use for making computations.

1.1.1 Definition and Structure of a Computing Medium

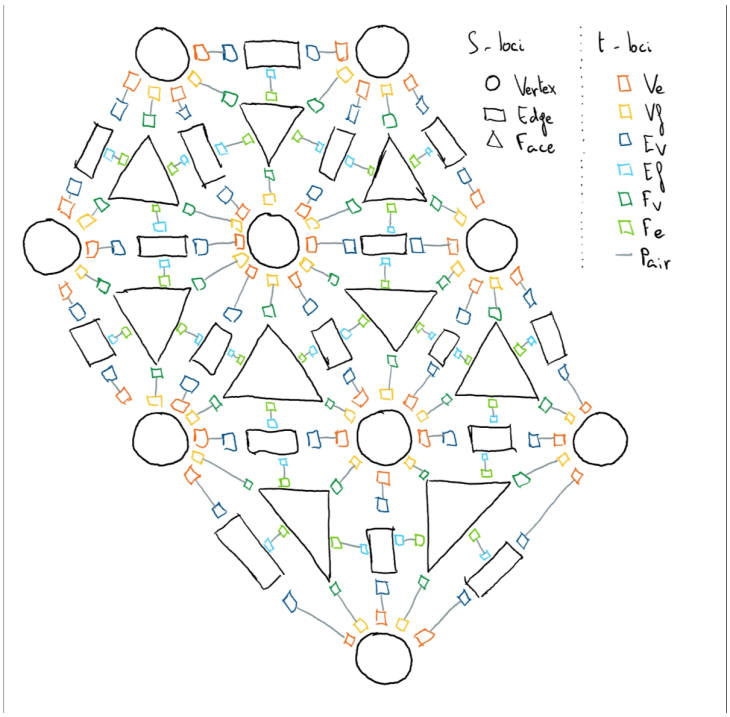


Fig. 1.1: Example of triangulated medium.

In its most general form, a computing medium is simply a set of processing elements distributed in some space which are locally connected. Here, we focus exclusively on a specific kind of media called "triangulated computing media", hence the term "computing medium" in the rest of this report will refer to those unless specified otherwise. A triangulated medium is represented as a Delaunay-triangulated graph where each vertex, edge,

and face is associated with a processing element capable of executing minimal computation, storing information, and communicating with its neighbors.

Vertices, edges, and faces are called "simplicial loci" (or "S-loci" for short). They determine the overall structure of the medium. Each S-locus controls a set of "transfer loci" (or "t-loci"). T-loci come in pairs that handle the communication between two S-loci. For example, if a vertex was connected to an edge, there would be a pair of t-loci (one belonging to the vertex and the other belonging to the edge) between them.

Obviously, there are 3 types of S-loci : Vertex, Edge, Face. T-loci are divided into 6 types :

- Ve linking a Vertex to an Edge
- Vf linking a Vertex to a Face
- Ev linking an Edge to a Vertex
- Ef linking an Edge to a Face
- Fv linking a Face to a Vertex
- Fe linking a Face to an Edge

Figure 1.1 shows an example of how all these loci are put together to form a medium. For reference, a vertex controls any number of Ve's and Vf's[1], an edge controls two Ev's and two Ef's, and a face controls three Fv's and three Fe's.

A medium is said to be crystalline if a repeating structure emerges in the placement if its vertices. It is amorphous when no such structure emerges. Additionally, it is homogeneous if the average point density is approximately constant and there are no "holes" or "clusters", and it is isotropic if the directions of the edges of the medium follow a uniform distribution over $[0, \pi[$. This report mainly focuses on amorphous homogeneous isotropic (AHI) media.

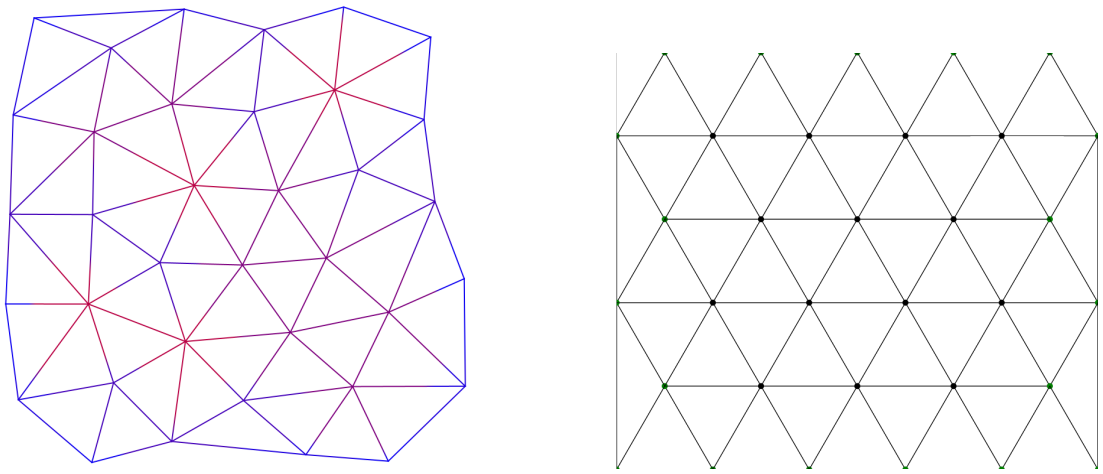


Fig. 1.2: An AHI medium (left) VS a crystalline hexagonal medium (right).

For now, we consider PEs to be immobile and synchronous, although lifting these two constraints will probably be the subject of future research.

Blobs form the computational basis of blob computing. A blob of a given medium M is a connected sub-graph M' of M such that all the vertices of M' share a common property (e.g.: all their values are even, or all their values are 1, etc.). Formally, it is a "True" connected component of a boolean field over the vertices (see [2]).

Importantly, blobs are persistent through time, meaning that two blobs at two different points in time can be considered one and the same under the right conditions, even though blobs are able to change their shape over time. This is possible for two reasons :

1. Blobs maintain their connectivity, they typically do not merge or divide unless instructed otherwise.
2. Blobs only change shape at their border : at each time step they can grow to unoccupied vertices adjacent to their border, or they can lose vertices that are part of their border.

Therefore, two blobs at two consecutive timestep are "the same blob" if they only differ at their border. We can then extend the notion to any number of timestep through its transitive closure.

1.1.2 An Example of Computation : the Voronoï Diagram

[2] develops tools to program on computing media. As an example, it shows how to build a (discrete) Voronoï diagram from a set of "seeds" in a medium.

[1] Although importantly, for a sufficiently homogeneous medium (see below), we conjecture that vertices that are not part of its border always control 4 to 8 Ve's and the same number of Vf's. This conjecture seems to hold experimentally.

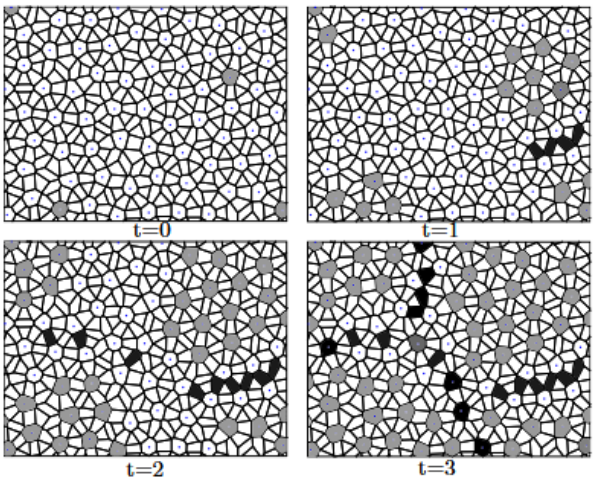


Fig. 1.3: Computation of a discrete Voronoï diagram.
Cells marked with a dot are the vertices of the medium. Gray cells form blobs. Black cells mark the frontiers between blobs. At t=0, blobs only cover the seeds. At t=3, blobs cover the Voronoï regions of the medium.
Source : [2]

The basic idea is the following : we start with a set of "seeds" distributed over the vertices of the network. Each seed is now the center of a blob. Then, at each synchronous step, the blobs will spread to their adjacent vertices if this would not cause them to merge with another blob, until no blob can spread further. It can be implemented with very few types of local operations that are applied simultaneously to every S-locus in the network :

- Multi-cast :
  The S-locus copies its value into its t-loci of a given type.
- Rotation :
  Since t-loci are spatially arranged around their S-loci, we can define a clockwise and counter-clockwise neighbor. T-loci can perform logical operations on their and their neighbors' values.
- Reduction :
  The S-locus stores the result of a commutative and associative operation applied to the values of its t-loci of a given type.
- Transfer :
  Each of the S-locus' t-loci exchanges its value with the value of its pair.

In fact, clever association of these primitives allows to program everything a computing medium is capable of, such as sorting arrays of integer or multiplying matrices[5].

1.1.3 Objective : Cannings

Up until now, Gruau mainly focused on hexagonal crystalline media. He built a plateform allowing efficient simulation of blob computation on classical hardware using SIMD[2,3].

The goal of this TER is to allow the use of SIMD to optimize the simulation of AHI media as well. We do this through the development of the notion of "total canning", or simply "canning". Given a medium M, a canning is a set of 9 injective maps which assign some integer coordinates to each locus of M :

1. $\{\text{Vertex}_M\} \mapsto \mathbb{N}^{a_{\text{V}}}$
2. $\{\text{Edge}_M\} \mapsto \mathbb{N}^{a_{\text{E}}}$
3. $\{\text{Face}_M\} \mapsto \mathbb{N}^{a_{\text{F}}}$
4. $\{\text{Ve}_M\} \mapsto \mathbb{N}^{a_{\text{Ve}}}$
5. $\{\text{Vf}_M\} \mapsto \mathbb{N}^{a_{\text{Vf}}}$
6. $\{\text{Ev}_M\} \mapsto \mathbb{N}^{a_{\text{Ev}}}$
7. $\{\text{Ef}_M\} \mapsto \mathbb{N}^{a_{\text{Ef}}}$
8. $\{\text{Fv}_M\} \mapsto \mathbb{N}^{a_{\text{Fv}}}$
9. $\{\text{Fe}_M\} \mapsto \mathbb{N}^{a_{\text{Fe}}}$

   where $a_{\text{V}}, ..., a_{\text{Fe}} \geq 1$ are specific to each canning method and designate the number of coordinates assigned to each type of locus. We call a_T the arity of type T under a given canning.

Informally, the term "canning" will designate either this set of maps, or the algorithm used to build it.

The objective is twofold : firstly, we want to find a way to measure the efficiency of a canning on a given medium. Secondly, we want to create a canning that is efficient on most -if not all- media.

1.2 Preliminary Work

In this section, I briefly introduce some work I did last year during another internship with Gruau to generate AHI media. While it is not the main focus of this year's TER, it is what made it possible in its current form.

1.2.1 Delaunay Triangulation

A Delaunay triangulation is a particular type of triangulation where a triplet of vertices form a face if and only if no other vertex can be found inside the circumcircle of the triplet. Normally, the edges forming the convex hull of the embedded graph would be part of the triangulation, however we will allow the removal of those edges under certain conditions as detailed below. This particular type of triangulation presents interesting properties for our purpose as the edges of the triangulation tend to remain "localized" (i.e. vertices that are connected tend to be spatially close).

I used the triangulation algorithm described in [6]. However, there is a bug in my implementation where vertically-aligned vertices are often handled incorrectly, which I could not fix before the end of my internship last year. Since this problem was not particularly consequential, I did not take the time to fix it this year either.

Additionally, the FPO algorithm (see 1.2.2) performs a lot of removals and insertions to the set of vertices. Currently, each time this happens, the entire set is retriangulated. This is a vastly inefficient approach to the problem. If I had the time, I would have liked to implement the removal and addition algorithms described in [6, 1].

1.2.2 Farthest Point Optimization

The Farthest Point Optimization algorithm[8] allows the construction of the irregular yet homogeneous sets of points that we need for blob computing. It yields better results than more common algorithm like Poisson-Disk sampling or Lloyd algorithm, and fits naturally into the project as it relies heavily on the Delaunay triangulation.

The implementation of FPO in my case was not straightforward. This is because the article applied the algorithm to sets of points in the unit torus, while the media uses bounded subsets of the Euclidean plane instead. In particular, I had to adapt the algorithm to work around the existence of borders with predetermined shape, and that can contain fixed points that cannot be moved even though they can still influence the placement of other points.

Chapter 2

TER

2.1 Borders and Media

The border of a medium is the set composed of both the edges separating the outer face of the graph from the inner faces of the graph, and the vertices at the ends of those edges.

The method we chose to handle the borders of our media is crucial, as it impacts how the media are optimized, how they are canned afterward, and decide some useful properties they have for performing computations. Ultimately, we decided to study two broad types of borders : "soft" and "hard" borders.

2.1.1 Soft Borders

Soft borders are the least restrictive of the two types. A medium has a soft border if all of its vertices are allowed to be moved anywhere in a bounded, connected region of the Euclidean plane during FPO. This region can take any shape.

This method of handling the border tends to connect vertices that would normally be too far apart from each other if they were inside the medium, breaking its locality. To solve this issue, we allow the iterative removal of the edges of the border that connects vertices which would otherwise be "too far apart".

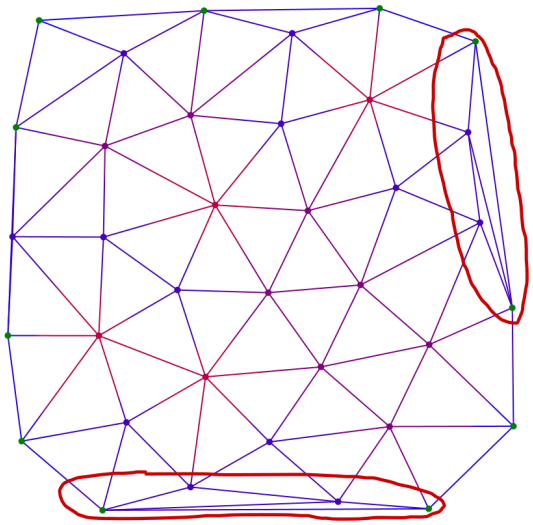


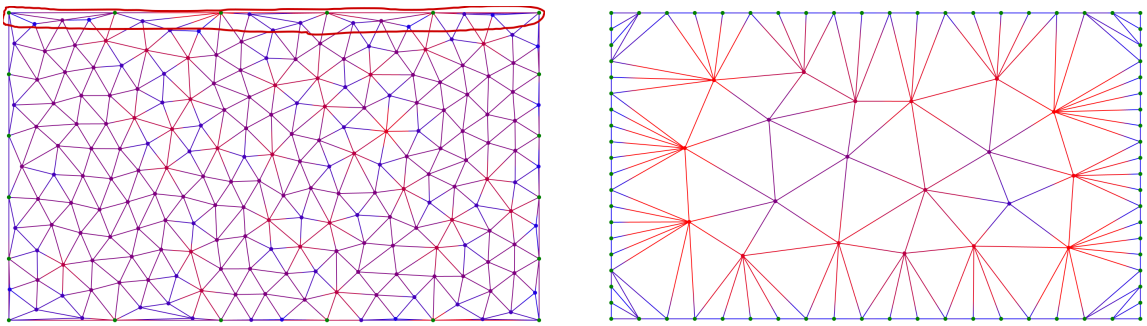
Fig. 2.1: Loss of locality without border correction.

We initially considered two types of soft bordered media : soft square media, where the vertices are confined into a square, and soft circle media, where the vertices are confined into a circle. However, we quickly abandoned soft circle media as they currently do not have any application for computing.

2.1.2 Hard Borders

A hard bordered medium is a media whose border (both the edges and the vertices) is fixed even before FPO. Hard borders are a lot more restrictive than soft borders, but they also offer many advantages over them. In particular, since we can decide the placement of the vertices of the borders, it means they can arranged in a way

that makes them easy to mirror or to fold into tori. This has interesting properties once the media is used for computation. However, one must be careful when constructing such media, as over/under-filling can lead to a loss of homogeneity or locality near the border.



(a) Loss of locality in over-filled medium.

(b) Loss of homogeneity in under-filled medium.

Fig. 2.2: Effects of over- and under-filling.

Initially, we tried to work with a border following a hexagonal lattice to approach previous work, but we ended dropping the idea as we would either experience the loss of homogeneity/locality mentioned above, or FPO would end converging back to a hexagonal crystal, losing the isotropicism of the media. Instead, we focused on two types of hard border : a hard square with evenly placed vertices, and a hard rectangle with ratio $1:\sqrt{3}$. We chose this ratio as it is the ratio of the minimal over maximal radius of a regular hexagon. This allows some semblance of hexagonal structure in the border while avoiding the problems we encountered with an outright hexagonal pattern.

2.1.3 About Hybrid Borders

We also briefly considered using a hybrid type of border by allowing the vertices of the borders to move in a restricted manner, we ultimately decided against it as it made our hypothesis more restrictive than pure soft borders while bringing non of the advantages that hard borders have.

2.2 Canning and Evaluation

Before we begin our search for total cannings, let us define the notion of partial canning. A partial canning of a medium M is simply a subset of a total canning of M (or the algorithm used to build it). In particular, we focus on vertex cannings, i.e. partial cannings which only contain the map from the vertices of M to their coordinates. From there, we will exhibit a generic method for build total cannings using vertex canning as a base. Finally, we will show how we can evaluate the efficiency of different cannings so we can compare them.

2.2.1 Vertex Cannings

To align with the way crystaline media are hendled, we set $a_V = 2$, i.e. we want to assign 2D integer coordinates to each vertex of the medium. The problem now becomes equivalent to placing the vertices in a 2D grid. The vertex canning is valid if all the vertices have been placed in the grid and if each cell of the grid contains at most one vertex. The coordinates of each vertex then becomes the 2D index of its cell. Based on this idea, I have developped two diffrent vertex cannings with opposite philosophies :

1. **TopDistanceXStorted** vertex canning
   This vertex canning was developed to rely as much as possible on the graph structure of the medium, making as little use as possible of its embedding. It was developed quite early in the project, before soft-bordered media appeared, so it only works on hard-bordered media. It works in the following way :
   - First, compute the minimum graph-distance of each vertex to any vertex on the topside of the border. All vertices at distance Y from the top are placed in line Y of the grid.
   - Second, place the leftmost vertex of each line in the first cell of its line, the second leftmost vertex of each line in the second cell of its line, repeating until all vertices are placed in the grid.
2. **RoundedCoordinates** vertex canning
   This vertex canning was developed to rely exclusively on the spatial embedding of the medium, completing ignoring its graph structure. It works in the following way :
   - First set a counter to 1.
   - Multiply the spatial coordinates of each vertex by the counter and round the result. Use the two integers obtained as a 2D index to put each vertex in the grid.

- If two vertices share the same cell, increment the counter by 1 and go back to the previous step. Otherwise, we have a vertex canning.

Regarding the TopDistanceXStorted algorithm, I first tried sorting the lines using the distance to the left-side border, however it resulted in invalid vertex cannings where two vertices would share the same cell.

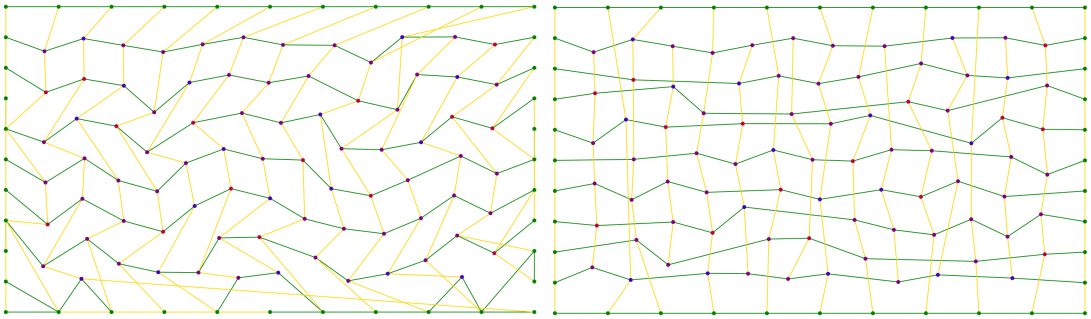


Fig. 2.3: TopDistanceXSorted (left) next to RoundedCoordinates (right) applied to the same medium. Two vertices are joint by a green line if they are consecutive on a line of the grid, and a yellow line if they are consecutive in a column.

Intuitively from figure 2.3, we can infer that RoundedCoordinates is better as it creates a more "grid-like" canning, whereas the columns in TopDistanceXSorted seem to "drift" to the left. We will confirm this intuition later.

2.2.2 From Vertex to Total Cannings

Now that we can build vertex cannings, we want a method to convert them into total cannings.

To do this, we first define the "belonging" relation between loci, such that we can say that some locus A belongs to some other locus B. We do this so that can "index" B relative to A. Given some vertex canning VC, we define the relation in the following way :

- Vertices belong to no one.
- We iterate over the lines of VC, going from the top-most line to the bottom-most line in order. Then we iterate over each vertex V of the line, going from the left-most to the right-most in order. Each Edge and Face connected to V now belong to V, unless it already belongs to another vertex.
- Each t-locus belongs to the S-locus that has control over it.

Now, given a vertex v, an edge e and a face f we write
- y_v and x_v the coordinates of v in VC,
- $E(v)$ the set of edges belonging to v,
- $F(v)$ the set of faces belonging to v,
- $Ve(v)$ the set of Ve t-loci belonging to v
- $Vf(v)$ the set of Vf t-loci belonging to v
- $Ev(e)$ the set of Ev t-loci belonging to e
- $Ef(e)$ the set of Ef t-loci belonging to e
- $Fv(f)$ the set of Fv t-loci belonging to f
- $Fe(f)$ the set of Fe t-loci belonging to f

Since the edges, faces, Ve's and Vf's will be indexed relative to vertices, we set the arities $a_{\mathrm{E}} = a_{\mathrm{F}} = a_{\mathrm{Ve}} = a_{\mathrm{Vf}} = a_{\mathrm{V}} + 1 = 3$. Similarly, Ev's and Ef's will be indexed relative to edges so we set $a_{\mathrm{Ev}} = a_{\mathrm{Ef}} = a_{\mathrm{E}} + 1 = 4$, and Fv's and Fe's will be indexed relative to faces so we set $a_{\mathrm{Fv}} = a_{\mathrm{Fe}} = a_{\mathrm{F}} + 1 = 4$. Now, the attribution of coordinates works in the following way :

- **Edges** :
  For each vertex v
  - Select a first edge.
    If it exists and it is in $E(v)$, it is the edge joining v to the next vertex in its line.
    Otherwise, start on the left of v and walk clockwise around it, selecting the first edge in $E(v)$ that you cross.
  - Assign each edge in $E(v)$ an integer between 0 and $|E(v)| - 1$, starting from the first edge and sorted clockwise around v.
  - Edge number p now has coordinates (p, y_v, x_v).
- **Faces** :
  For each vertex v

- Select a first face.
  If it exists and it is in $F(v)$, it is the face joining v to the next vertex in its line and to v's next neighbor clockwise.
  Otherwise, start on the left of v and walk clockwise around it, selecting the first face in $F(v)$ that you cross.
- Assign each face in $F(v)$ an integer between 0 and $|F(v)| - 1$, starting from the first face and sorted clockwise around v.
- Face number p now has coordinates (p, y_v, x_v).

- **Ve's** :
  For each vertex v
  - Select a first Ve. It is the Ve linking to v's first edge.
  - Assign each Ve in $Ve(v)$ an integer between 0 and $|Ve(v)| - 1$, starting from the first Ve and sorted clockwise around v.
  - Ve number p now has coordinates (p, y_v, x_v).
- **Vf's** :
  For each vertex v
  - Select a first Vf. It is the Vf linking to v's first face.
  - Assign each Vf in $Vf(v)$ an integer between 0 and $|Vf(v)| - 1$, starting from the first Vf and sorted clockwise around v.
  - Vf number p now has coordinates (p, y_v, x_v).
- **Ev's** :
  For each Edge e
  - We note (p, y_e, x_e) its coordinates.
  - The Ev linking to the vertex to which e belongs has coordinates $(0, p, y_e, x_e)$.
  - The other Ev has coordinates $(1, p, y_e, x_e)$.
- **Ef's** :
  For each Edge e
  - We note (p, y_e, x_e) its coordinates, and v the vertex it belongs to.
  - We consider that e "faces" the vertex to which it belongs, which let us distinguish between a right Ef and a left Ef.
  - The right Ef has coordinates $(0, p, y_e, x_e)$.
  - The left Ef has coordinates $(1, p, y_e, x_e)$.
- **Fv's** :
  For each face f
  - We note (p, y_f, x_f) its coordinates, and v the vertex it belongs to.
  - We call "first Fv" the Fv linking to v
  - Assign each Fv in $Fv(f)$ an integer between 0 and 2, starting from the first Fv and sorted clockwise around f.
  - Fv number q now has coordinates (q, p, y_f, x_f)
- **Fe's** :
  For each face f
  - We note (p, y_f, x_f) its coordinates, and v the vertex it belongs to.
  - We call "first Fe" the Fe clockwise-next to the Fv linking to v
  - Assign each Fe in $Fe(f)$ an integer between 0 and 2, starting from the first Fe and sorted clockwise around f.
  - Fe number q now has coordinates (q, p, y_f, x_f)

2.2.3 Evaluation

Definition

Finally, now that we have total cannings, we want a way to evaluate them. First we remember that the goal of a canning is to allow the use of SIMD to simulate the execution of a computing media. Therefore, we can decide how good a canning is by deciding how many operations are needed to simulate each an execution step, and how well they can be parallelized. While it is possible to compute this exactly, for now we only focus on finding an upper bound. We do this by noticing that since

1. each t-locus in a canning is indexed relative to a vertex and
2. all communications are uniquely determined by the triangulated graph, which is itself (with probability 1 in general) uniquely determined by the placement of the vertices,

we can find an upper bound for each communication type with a single computation over the canning of the vertices, which will be accurate up to a constant factor that depends on the type of communication[1].

This upper bound is given by a Δy, which designates the maximum distance between two lines of vertices that interact with each other, and a Δx, which designates the maximum distance between two columns of vertices that interact with each other. Together, they allow computing $\mathcal{A}_{max} = (2\Delta y + 1)(2\Delta x + 1)$ the maximum area of interaction around any vertices.

Together with the constant factor we defined earlier, this $\mathcal{A}_{max}$ forms the upper bound were looking for. Therefore $\mathcal{A}_{max}$ is a suitable measurement of the performance of a canning, which we wish to minimize.

Some results

Overall, as we guessed earlier, the RoundedCoordinates algorithm yields much better results than the TopDistanceXSorted algorithm. RoundedCoordinates produces an $\mathcal{A}_{max}$ oscillating between 25 and 49 regardless of the size the medium. Meanwhile TopDistanceXSorted produces an $\mathcal{A}_{max}$ that grows with the size of the medium. It outperforms RoundedCoordinates for very small media (less than 50 vertices) but quickly becomes much worse with an $\mathcal{A}_{max}$ reaching upward of 150 for 1000 vertices, and upward of 200 for 2000 vertices.

RoundedCoordinates seems to perform overall better on (reasonably defined) hard bordered media. This aligns with our observation that soft borders join vertices that would normally be too far apart from each other, perhaps indicating that we are too lenient when initially correcting the border.

TopDistanceXSorted seems to get worse the wider the media get. This is further evidence that the "drift" in the columns we observed earlier is responsible for the poor performance of this method.

[1] For example, there is exactly 2 Ev per edge, and there are at most 3 times as many edges as there are vertices, so Ev-Ve communication will have a constant factor of 6. This factor can be seen as the density of a given type of locus relative to the vertices.

Chapter 3

Internship

3.0 Some conventions going forward

We decided to adopt some convention to reduce the scope of the research.

- We decided to work only on media with a hard rectangular border, as they are much easier to work with and present properties for the simulation (most notably, they can be mirrored along their sides or repeated in every direction in a grid-like pattern).

- We only work on vertex cannings, and rely entirely on the algorithm described in 2.2.2 to convert them into total vertex cannings.

3.1 Platform-CA2[3] and Evaluation

I spent the first two weeks of the internship reading the source code of the simulation platform at [3]. I was hoping to be able to implement the simulation of amorphous media into it, although we ultimately decided against it as the internship was simply too short for this to be a realistic goal.

So how does this platform work, and how exactly do cannings help ?

3.1.1 How is SIMD applied ?

To simplify things, let's assume that vertices can communicate between themselves directly without the need for transfer locus intermediary edges, and that they each contain a boolean value. We will now take the example of an OR reduction on the third line of vertices of a medium M. It is a simple operation : each vertex has (up to) 8 neighbors, and we compute its new value as $Value(neighbor_0) \vee \cdots \vee Value(neighbor_7)$.

First, we fill two 2D arrays : one that will contain the starting values of the reduction, i.e. the values contained in the vertices before the reduction, and the other will be our result table which and will contains the new values of our vertices after the reduction. These arrays are linked to a canning C of M in that the value in the cell (Y, X) correspond to the value of the vertex at position (Y, X) in C.

Values

L0	0	0	1	1	1	0
L1	1	0	1	0	1	1
L2	0	0	0	1	1	0
L3	1	0	0	0	1	0

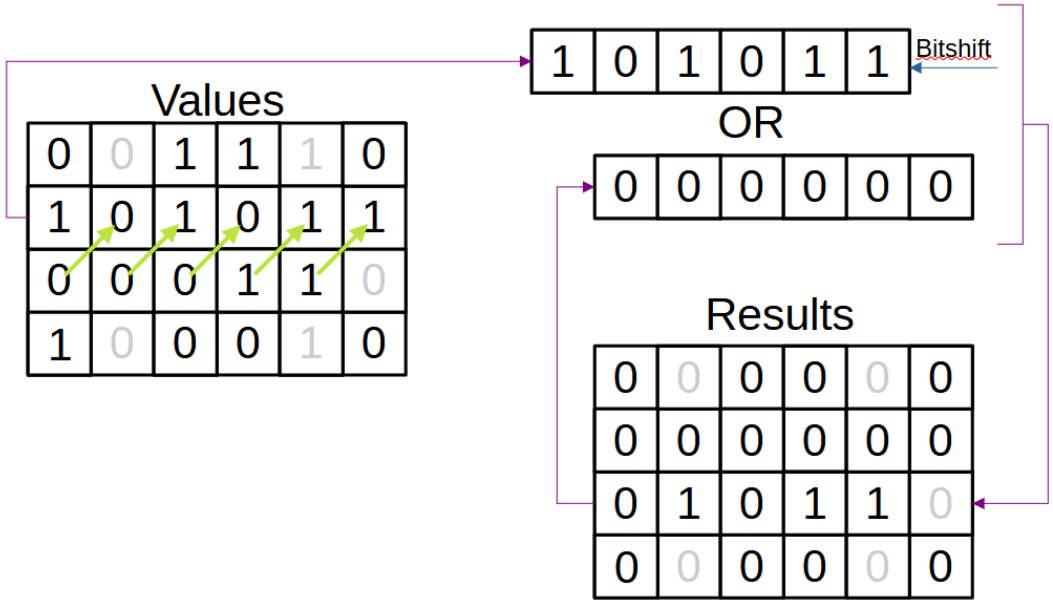
Results

0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Since 0 is the neutral element of the OR operator, we use it to initialize our result table. Grey values in the grid indicate that the cell does not correspond to a vertex, which means it contains a garbage value that can be safely ignored as it will never be used.

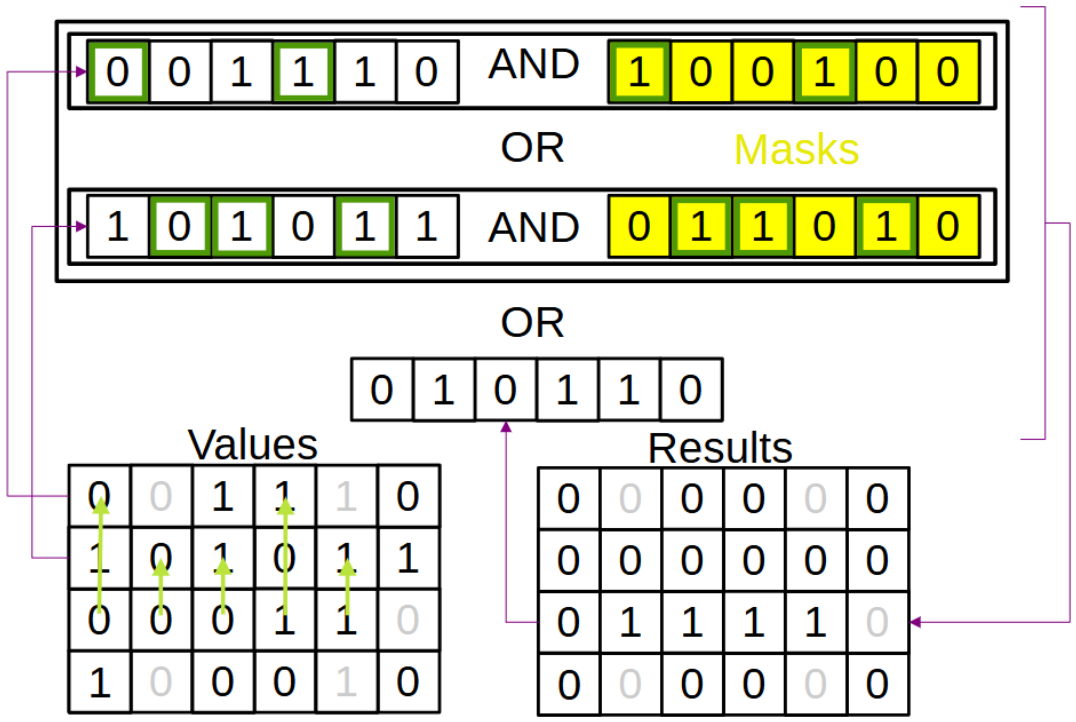
An here is where the magic operates : instead of storing $L0$ as an array of boolean values containing $\{0, 0, 1, 1, 1, 0\}$, we store it as the integer $0b001110$, or simply 14. This allows us to perform our simulation much faster, as we can now apply an operation (such as a bitwise OR) on an entire line of vertices in a single action.

Now, let's perform the first step of our reduction :



Here, we notice the first neighbor of each vertex is stored in the cell directly to its top right. Since the neighbors are one step to far to the right, we want to shift them back one step left to realign them vertically with our vertices before OR'ing the line to the values currently stored in our result table.

However, this situation where everything works perfectly with just a single bitshift would probably never happen. What if instead not all vertices on the line had their i-th neighbor in the same position relative to them ? Let's perform the second step of our reduction :



First, we select all the lines that contain a second neighbor of a vertex on the third line. Note that a line can be selected multiple times if reaching the appropriate cells requires different bitshifts. Then, we apply a mask to each line selected to destroy all the information that we don't need so that it doesn't interfere with the result. Finally, we recombine all these lines by consecutively OR'ing them to to the values currently stored in our result table.

Now, in reality, vertices don't communicate directly but the two concepts above apply no matter which types

of loci are communicating as they are all stored as n-dimensional arrays with the last dimension being represented as an integer instead of an actual array.

3.1.2 Exact Measurement of the Performance of a Canning

From there, establishing an exact method of measuring the performance of canning might seem easy : the best canning is the one that minimizes 4 values :

1. The number of lines in the canning
2. The number of fragments of length $32 - 2\Delta x$ in each line (i.e. the Δx)
3. The number of masks per fragment

Additionally, we also want to minimize the Δy as it determines how many lines must remain readily accessible in memory at any given point in time.

However, it is important to notice that different actions need different amounts of masks. For example, an Ev→Ve communication might not require as many masks as a Fe→Ef communication. Similarly, a Vertex to 1st Ve communication might not require as many masks as a Vertex to 8th Ve communication. Still, all symmetrical communications use the same amount of masks.

Most importantly, this means that it may not be possible to the best canning for a given medium, as the effectiveness of the canning will be dependent on the tasks performed.

3.1.3 SIMD limitations

A key limitation of this approach to SIMD is the size of the "int" type. In java, an int is stored as a sequence of 32 bits. This means that a single integer cannot store a full line if it contains more than 32 booleans, which has two consequences :

1. Lines longer than 32 nits need to be chopped into int arrays, and there need to be a way to "reattach" them. In the current implementation of the the simulator, this is done by reserving Δx (as defined in 2.2.3) bits on each side of the integer as this guarantees that all the ints can be bitshifted in either direction without losing any information. For $\Delta x = 2$, this only leaves 28 usable bits per int to store information.
2. Since each int encodes information for $32 - 2\Delta x$, we have a maximum speedup of $32 - 2\Delta x$. However, each mask needed to apply a computation to a line fragment decreases that speedup rather significantly. To continue with our OR reduction example : let's imagine a canning with $\Delta x = 2$, and a line fragment associated with 3 masks. Then, to update our result table, we need 9 bitwise operations : 3 "bithshifts" to align our data, 3 "and" to apply our masks, and 3 "or" to update our results. This means that the total speedup is $\lfloor 28/9 \rfloor = 3$, far from the factor of 32 we hoped for. This also means that there is a rather low upper limit on the amount of masks that can be used before this method starts hindering the performance of the simulator instead of improving it.

Still, even though it is difficult to significantly improve the running time, this methods provides two other benefits : it allows to reuse the same program for both crystalline and amorphous media without much changes, and it still improves memory usage by a factor of $32 - 2\Delta x$ as the additional space required to store the masks quickly dwarfs in front of the space needed to store the fields.

3.1.4 Importance of Δ's and Density

As we just saw, the Δx is another important factor on the performance of a canning, as is indicates how much information can be stored in a line fragment. The Δy is also important, as it determines how many lines need to be loaded in memory at any time. It is sufficient to compute them on the vertex canning alone as it fully determines the entire canning.

The last performance metric we use is the density of a vertex canning. It is the ratio of the number of vertex in the medium over the number of cells in the vertex canning. It is an interesting metric because it is very efficient to compute, and it has many different uses.

Most obviously, increasing the density lowers the space between the vertices and is therefore correlated to reducing the Δ's *up to a certain point*. However, increasing the density too much starts to cause vertices to get "too close" to each other, which lowers the performance of the canning. This happens because vertices become harder to place near there neighbors as the amount of empty cells in the grid diminishes, and conflicts start to appear when two (or more) vertices try to occupy the same cell without a good way to relocate one of them.

On the other hand, decreasing the starting density *up to a certain point* often yields better result when the canning is optimized through simulated annealing, because the algorithms we use tend to rely on moving vertices in the grid to empty adjacent cells which is made easier when many cells are empty. But, like before, decreasing the density too much becomes detrimental, as the annealing starts allocating a lot of resources just to reduce the density in the first place, without much benefit afterward.

In the end, we have a lot of forces trying to pull the density in every direction, which makes it a parameter that is both very interesting and hard to use correctly.

3.2 Creating Vertex Cannings From Scratch

3.2.1 Revisiting the Rounded Coordinates Algorithm[2.2.1]

Although the algorithm seems to work well, one key issue with its design is the arbitrariness of the increment. Initially, it was always one, but there is no reason to that particular value except for ease of programming.

Firstly, we want to notice that the objective of this algorithm is to find a suitable "scaling factor" by which we can multiply our coordinates, such that they remain unique even after rounding. The increment method was simply a way to find such scaling factor.

One way to remove the arbitrariness is to perform a dichotomic search for the minimum working scaling factor. However, this methods yields terrible results (in fact, often worse than simply setting the increment to one). This is caused by the fact that a scaling factor x_1 can work while being smaller than a scaling x_2 which doesn't, making the dichotomic search ill-suited for this problem. This method had the additional drawback of trying to reduce increase the density of the canning at all cost, even if it would cause it to hurt the overall performance of the canning.

A better way to get rid of the arbitrariness is to use simulated annealing to optimize the value of the increment. This method allow for a thorough search over the increments (provided that the neighbor generation allows to reach any real number), while actually allowing to optimize the canning based on its performance, rather than simply its density.

While it may seem like a good idea to optimize the scaling factor directly rather than the increment, there is a pitfall to be aware of : it is very difficult to determine a good search interval, as the final value of the scaling is very sensitive to both the size of medium and its shape. Optimizing the increment instead basically guarantees that 1 is a good starting position regardless of the medium, and that a good solution will be found relatively close by.

However, even with the arbitrariness gone, we run into a second issue : this algorithm performs really well on square media, but struggles with rectangular media. This problem arises because we use the same scaling factor for both the y and x coordinates : since the rectangle is longer than it is tall (or vice-versa), we require a larger horizontal scaling factor (i.e. we want longer lines to fit all our vertices), but because we use the same scaling factor for both coordinates, we also en up with taller, emptier columns. The solution is then easy : using two separated scaling factors.

Once this correction is implemented, the algorithm starts performing almost as well on rectangular grids as it does on square grids.

3.2.2 A Divide and Conquer Algorithm

A divide and conquer algorithm also seems attractive on paper, since our objective is to build a grid around our vertices, but it is difficult to implement in practice.

The first difficulty is to know when to stop the division. Fortunately, it is relatively easy to computed an "expected length" for the lines and columns of the canning : given medium with n vertices, width w and height h, a line should contain close to $\lceil \frac{w}{h}\sqrt{n} \rceil$ vertices, and a column close to $\lceil \frac{h}{w}\sqrt{n} \rceil$ vertices.

Now, we could be tempted to define the lines (resp. columns) by recursively dividing the medium in half until all the lines (resp. columns) contain at most the expected number of vertices, but causes several issues :

- This systematically results in cannings whose number of lines and columns are powers of 2, which may not always be well-suited.
- Two vertices could both be placed in the same line and the same column. This means that they would be placed in the same cell, which is disallowed by definition.
- This would create lines and columns that are extremely uneven.

Instead, the current algorithm work by that an expected number of vertices on a line (resp. column) is also an expected number of columns (resp. lines) in the canning. Then it recursively divides the medium to reach that expected number in the following manner :

1. If there is one column (resp. line) to create Put the expected amount of vertices in it, starting from the closest to the center of the current part.
2. If there is a pair number of columns (resp. lines) to create
   Divide the medium into two parts with half the vertices on the left (resp. above) and half the vertices on the right (resp. below), and repeat this algorithm on each part.
3. If there is odd number of columns (resp. lines) to create
   Create a column (resp. line) and fill it with the expected amount of the vertices that are closest to the middle of the current part. Repeat this algorithm on the the two parts that are left on each side of side of the new column (resp. line).

This approach solves the problem of having uneven lines and columns and provides number of lines and columns that more fitted to the medium, but doesn't solve the issue of vertices sharing the same cell and even result in vertices that are not placed at all. Placing those missing vertices correctly while separating those which share the same cells is not trivial at all, and ultimately I couldn't find a goo way to do it.

3.3 Optimizing Existing Vertex Cannings

Once a canning has been created, it may be beneficial to modify it. For example, should a canning contain an empty line, there is no reason not to remove it.

3.3.1 Adaptative Grid Method

This method tries to optimize cannings whose lines and columns are separable, i.e. it is possible to draw straight horizontal and vertical lines that mark a separation between all the lines and columns of the the canning. It aims to reduce the deltas of such cannings by merging lines and columns wherever there is a "problematic" pair of neighbors.

For example, a canning with a Δx of 4 means means that there is at least one pair of vertices that are neighbor and separated by 3 columns (covering 5 in total). This method then attempts to merge those 5 columns into 4, thus reducing the Δx to 3. To do this, the method checks that on each line of the canning, at least one of the columns contains an empty cell. If this is the case, then merging is easy. Otherwise, the method allows vertices to move to an adjacent cell if they close enough to the border of that cell to try and create an empty cell on each line. This method keeps going as long as there remain problematic pairs that it hasn't already tried to solve.

3.3.2 Simulated Annealing

Another approach is to optimize the vertex placements in the canning through simulated annealing.

Search Space and Neighbor Selection

Annealing Process

Conclusion

Note : the following conclusions are based on the results available in the file src/main/java/test/testResults/canningStats.csv

Over the course of this internship, I explored the notion of canning in depth.

Overall, the best method was to apply a simulated annealing algorithm to the increment of a Rounded Coordinates Incremental algorithm. For square medium, this method largely outperforms all others, even when using a single scaling factor. For rectangular, the dual scaling factor version remains usually better, although using a single factor and correcting the result with an Adaptive Grid algorithm may sometimes result in a better canning.

Disappointingly, these methods do not always allow to reach a maximum delta of 2 on large mediums ($\geq$ 2025 vertices), however I do strongly believe that such a result is always attainable, given how little problematic pairs of vertices there even in the largest grids. In particular, I believe that the Local Resolution methods has a strong potential.

A simulated annealing algorithm based on moving vertices in the canning should probably work too given enough time and resources, but it would need a lot more time to fine tune than I was able to spend. To make it work, one would need either a much faster computer than I had access to, or to completely rework the solution space exploration. I do not think that this direction is worth pursuing with my resources. Still, since this approach is highly parallelizable, having access to a computing cluster would make it a lot more attractive.

Similarly, I think Divide and Conquer methods face too many issues to be worth pursuing.

Finally, while I would've liked to test my methods on much larger grids (4096 vertices is still low), the time complexity of my FPO implementation simply did not allow it. In the future, it may be worth it to use grids generated through Poisson Disc Sampling. Even though this method will result in lower quality media (and thus deteriorate the results), it could still give us an idea of how well the methods developed here scale with the number of vertices.

Appendix : Use the application

A How to use the GUI

Upon launching the application, the user will be faced with the main window. It divided into 5 parts :

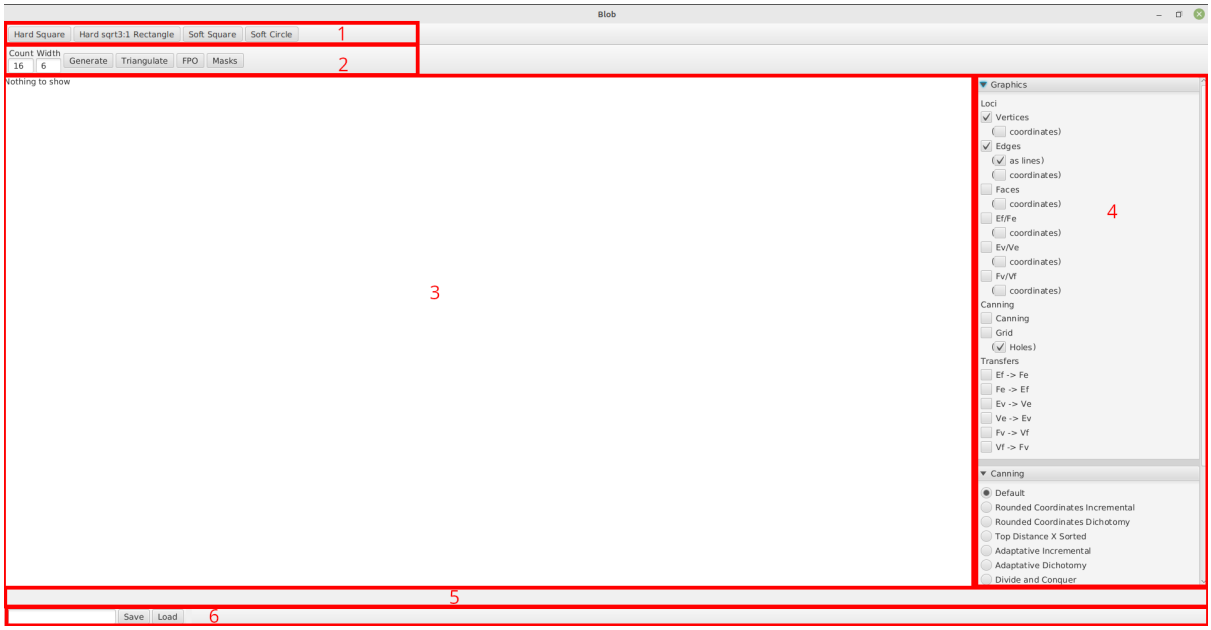


Fig. 3.1

1. **Medium shapes**
   This bar allows to chose the medium shape to use.
2. **Medium generation and optimization**
   This bar contains :
   (a) A series of user input fields that is defined separately for each medium shape.
   (b) A "Generate" button which creates a new random medium using the user-provided parameters.
   (c) A "Triangulate" button which applies a Delaunay triangulation of the medium.
   (d) An "FPO" button which both triangulates the medium and apply the FPO algorithm using the parameter provided the parameter panel.
   (e) A "Masks" button, which opens a new window displaying information about the masks needed to perform a communication using the current canning.
3. **Main view**
   This part displays the currently shown medium and canning.
4. **Parameter panel**
   This panel is used to change various parameters.
   The "Graphics" category impacts how the currently shown medium and canning are displayed
   The "Canning" category changes which canning should be displayed. Note that a canning is only computed when it is selected, and is recomputed every time the medium changes and every time it is re-selected after switching back from another canning. This operation can be time consuming, although the "Default" canning is usually very fast.
   The FPO category changes how the the FPO should be applied. The "Set iteration" mode performed as

many iteration of the algorithm as indicated in the textbox directly underneath it. The "To convergence" mode keep going until the medium is "sufficiently" optimized (as defined in the textbox directly underneath it, the closer to 1 the better), or until no further progress is made.
⚠The "To convergence" mode is bugged. Using 7 iterations should be enough for virtually any medium.

5. **Information bar**
   This bar displays various information about the currently shown medium an canning.
6. **Save management**
   This bar is used to save the currently shown medium and canning or load previously saved ones.
   To save a canning, write "<name>" in the textbox and click on the "save" button. This will create a new file (or overwrite an old file with the same) in the "save" directory called "<name>.vtxs".
   To load a canning, make sure a "<name>.vtxs" file exists in the "save" directory, write "<name>" in the textbox and click the "load" button.

B How to add new medium shapes

Start by creating a new class extending either `SoftBorderedMedium` or `HardBorderedMedium` (or `Medium` directly). In the case of a `HardBorderedMedium` extension, it is recommanded to override the `delaunayTriangulate` method to repair any incorrect link between vertices on the border that may occur after triangulation.

This class should contain at least one constructor filling the medium with randomly generated vertices (and not simply a list of preexisting vertices)

Create a new class extending `SavefileManager`. It should only override the `makeMedium` method with `public <MediumClass> makeMedium() { return new <MediumClass>(); }`

Create a class extending `MediumApp`. This class should define a constructor creating all the input you may need to add to the top tool bar (2 in figure 3.1), and create an instance of the suitable `SavefileManager` subclass. This class should also override the `generate` method to create the new medium according to user specification.

C How to add new cannings

To create a standalone canning :
Simply create a class implementing the `VertexCanning`. Its instances can later be passed to a `VertexCanningCompleter` to convert them to total cannings.

While it is possible to create a class directly implementing the `Canning` interface, this approach is not recommended, as the backup of such cannings is currently not supported.

To create an annealing-based canning :
Create a new class extending `Annealer<VertexCanning>`, and pass its instances to a `VertexCanningAnnealer` along with a `VertexCanning` to use as a seed for the annealing process. Finally, give the instance of the `VertexCanningAnnealer` to a `VertexCanningCompleter`.

To add a canning to the GUI :
In the source code of the class `MediumApp`, after the line `ToggleGroup canGroup = new ToggleGroup();` in `fillSidePanel()` :

1. Create a new `RadioButton rd` and add it to the toggle group with `rd.setToggleGroup(canGroup)`.
2. In the lambda passed to `canGroup.selectedToggleProperty().addListener()`, add

   ```
   else if (newVal == rd) { canningFactory = m -> new NewCanning(parameters...) }
   ```

3. Add `rd` to the constructor of `VBox canning`.

Bibliography

[1] Olivier Devillers. "On Deletion in Delaunay Triangulations". In: *International Journal of Computational Geometry and Applications* 12 (2002), pp. 193–205. DOI: 10.1142/S0218195902000815. URL: https://inria.hal.science/inria-00167201.

[2] Frédéric Gruau. "A parallel data-structure for modular programming of triangulated computing media." In: *Natural Computing* 22.4 (2023), pp. 753–766. ISSN: 1572-9796. DOI: 10.1007/s11047-022-09906-1. URL: https://doi.org/10.1007/s11047-022-09906-1.

[3] Frédéric Gruau. *platform-CA2*. URL: https://github.com/fredgruau/platform-CA2.

[4] Frédéric Gruau et al. "BLOB computing". In: *Proceedings of the 1st Conference on Computing Frontiers*. CF '04. Ischia, Italy: Association for Computing Machinery, 2004, 125–139. ISBN: 1581137419. DOI: 10.1145/977091.977111. URL: https://doi.org/10.1145/977091.977111.

[5] Frédéric Gruau, Christine Eisenbeis, and Luidnel Maignan. "The foundation of self-developing blob machines for spatial computing". In: *Physica D: Nonlinear Phenomena* 237.9 (2008). Novel Computing Paradigms: Quo Vadis?, pp. 1282–1301. ISSN: 0167-2789. DOI: https://doi.org/10.1016/j.physd.2008.03.046. URL: https://www.sciencedirect.com/science/article/pii/S0167278908001255.

[6] Leonidas Guibas and Jorge Stolfi. "Primitives for the manipulation of general subdivisions and the computation of Voronoi". In: *ACM Trans. Graph.* 4.2 (Apr. 1985), 74–123. ISSN: 0730-0301. DOI: 10.1145/282918.282923. URL: https://doi.org/10.1145/282918.282923.

[7] Iñaki Navarro and Fernando Matía. "An Introduction to Swarm Robotics". In: *International Scholarly Research Notices* 2013.1 (2013), p. 608164. DOI: https://doi.org/10.5402/2013/608164. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.5402/2013/608164. URL: https://onlinelibrary.wiley.com/doi/abs/10.5402/2013/608164.

[8] Thomas Schlömer, Daniel Heck, and Oliver Deussen. "Farthest-point optimized point sets with maximized minimum distance". In: *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics*. HPG '11. Vancouver, British Columbia, Canada: Association for Computing Machinery, 2011, 135–142. ISBN: 9781450308960. DOI: 10.1145/2018323.2018345. URL: https://doi.org/10.1145/2018323.2018345.